Digital Design Diploma

Mini Project : Spartan6 DSP48A1

Name:	**Roaa Atef**

Questasem Part:

1. Verilog code:

```v
module reg_plus_mux (clk,rst ,reg_en,D,D_MUX);
parameter PIPELINE =0;
parameter WIDTH=1;
parameter RST_TYPE="SYNC";
////////////////////
input clk,rst;
input [WIDTH-1:0]D;
input reg_en;
output [WIDTH-1:0]D_MUX;
reg [WIDTH-1:0] D_FF;
////////////////////////////////////////////////
assign D_MUX =(PIPELINE==0)?D:D_FF;
generate
if (RST_TYPE=="SYNC") begin
always @(posedge clk) begin
if(rst)
D_FF<=0;
 else if((reg_en))
begin
D_FF<=D;
end
end
end
else begin
always @(posedge clk or posedge rst ) begin
if(rst)
D_FF<=0;
 else if((reg_en))
begin
D_FF<=D;
end
end
end
endgenerate
endmodule

```

```v

module DSP (
input [17:0] A,
input [17:0] B,
input [47:0] C,
input [17:0] D,
input CARRYIN,
output [35:0] M,
output [47:0] P,
output   CARRYOUT,
output  CARRYOUTF,
input CLK,
input [7:0]OPMODE,
input CEA,CEB,CEC,CECARRYIN,CED,CEM,CEOPMODE,CEP,
input RSTA,RSTB,RSTC,RSTCARRYIN,RSTD,RSTM,RSTOPMODE,RSTP,
input[47:0] PCIN,
output [47:0] PCOUT,
output [17:0] BCOUT,
input [17:0] BCIN
    );
////////////////////////////////////////////////////
parameter A0REG=0;
parameter A1REG=1;
parameter B0REG=0;
parameter B1REG=1;
parameter CREG=1;
parameter DREG=1;
parameter MREG=1;
parameter PREG=1;
parameter CARRYINREG=1;
parameter CARRYOUTREG=1;
parameter OPMODEREG=1;
parameter CARRYINSEL="OPMODE5";
parameter B_INPUT="DIRECT";
parameter RSTTYPE="SYNC";
////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////
wire [17:0] A_reg;
wire [17:0] B_reg;
wire [47:0] C_reg;
wire [17:0] D_reg;
wire [17:0] A_reg2;
wire [17:0] B_reg2;
reg cout;
wire CARRYOUT_reg;
wire CARRYIN_reg;
reg CARRYIN_mux;
reg [17:0] B_mux;
wire [7:0] OPMODE_reg;
wire [35:0] M_reg;
wire [47:0] P_reg;
////////////////////////////////
reg [17:0] B_D_ADD_SUB;
reg [17:0] op_mode_4_mux;
reg [35:0] MUL;
```

```
reg [47:0] MUX_x;
reg [47:0] MUX_z;
reg [47:0] z_x_ADDER;
//////////////////////
assign BCOUT= B_reg2;
assign M=M_reg;
assign P=P_reg;
assign PCOUT=P_reg;
assign CARRYOUT=CARRYOUT_reg;
assign CARRYOUTF=CARRYOUT_reg;
//////////////////////
generate
if (A0REG==0) begin
    reg_plus_mux   #(.PIPELINE(A0REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_a (CLK,RSTA ,CEA,A,A_reg);

end
else begin
        reg_plus_mux   #(.PIPELINE(A0REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_a (CLK,RSTA ,CEA,A,A_reg);
end

//////////////////////////////////////////////////////

if (B0REG==0) begin
    reg_plus_mux   #(.PIPELINE(B0REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_b (CLK,RSTB ,CEB,B_mux,B_reg);

end
else begin
        reg_plus_mux   #(.PIPELINE(B0REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_b (CLK,RSTB ,CEB,B_mux,B_reg);
end

//////////////////////////////////////////////////////

if (CREG==0) begin
    reg_plus_mux   #(.PIPELINE(CREG),.WIDTH(48),.RST_TYPE(RSTTYPE)) reg_c (CLK,RSTC ,CEC,C,C_reg);

end
else begin
        reg_plus_mux   #(.PIPELINE(CREG),.WIDTH(48),.RST_TYPE(RSTTYPE)) reg_c (CLK,RSTC ,CEC,C,C_reg);
end

//////////////////////////////////////////////////////////

if (DREG==0) begin
    reg_plus_mux   #(.PIPELINE(DREG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_d (CLK,RSTD,CED,D,D_reg);

end
else begin
        reg_plus_mux   #(.PIPELINE(DREG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_d (CLK,RSTD ,CED,D,D_reg);
end

////////////////////////////////////

if (A1REG==0) begin
    reg_plus_mux   #(.PIPELINE(A1REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_a1 (CLK,RSTA ,CEA,A_reg,A_reg2);

```

```
end
else begin
        reg_plus_mux   #(.PIPELINE(A1REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_a1 (CLK,RSTA ,CEA,A_reg,A_reg2);
end

////////////////////////////////////////////////////

if (B1REG==0) begin
    reg_plus_mux   #(.PIPELINE(B1REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_b1 (CLK,RSTB ,CEB,op_mode_4_mux,B_reg2);

end
else begin
        reg_plus_mux   #(.PIPELINE(B1REG),.WIDTH(18),.RST_TYPE(RSTTYPE)) reg_b1 (CLK,RSTB ,CEB,op_mode_4_mux,B_reg2);
end

////////////////////////////////////////////////////

if (MREG==0) begin
    reg_plus_mux   #(.PIPELINE(MREG),.WIDTH(36),.RST_TYPE(RSTTYPE)) reg_m (CLK,RSTM ,CEM,MUL,M_reg);

end
else begin
        reg_plus_mux   #(.PIPELINE(MREG),.WIDTH(36),.RST_TYPE(RSTTYPE)) reg_m (CLK,RSTM ,CEM,MUL,M_reg);
end

////////////////////////////////////////////////////////

if (PREG==0) begin
    reg_plus_mux   #(.PIPELINE(PREG),.WIDTH(48),.RST_TYPE(RSTTYPE)) reg_p (CLK,RSTP,CEP,z_x_ADDER,P_reg);

end
else begin
        reg_plus_mux   #(.PIPELINE(PREG),.WIDTH(48),.RST_TYPE(RSTTYPE)) reg_p (CLK,RSTP ,CEP,z_x_ADDER,P_reg);
end

////////////////////////////////////////////////////

if (CARRYINREG==0) begin
    reg_plus_mux   #(.PIPELINE(CARRYINREG),.WIDTH(1),.RST_TYPE(RSTTYPE)) reg_carr (CLK,RSTCARRYIN ,CECARRYIN,CARRYIN_mux,CARRYIN_reg);

end
else begin
        reg_plus_mux   #(.PIPELINE(CARRYINREG),.WIDTH(1),.RST_TYPE(RSTTYPE)) reg_carr (CLK,RSTCARRYIN ,CECARRYIN,CARRYIN_mux,CARRYIN_reg);
end

////////////////////////////////////////////////////////

if (CARRYOUTREG==0) begin
    reg_plus_mux   #(.PIPELINE(CARRYOUTREG),.WIDTH(1),.RST_TYPE(RSTTYPE)) reg_carro (CLK,RSTCARRYIN,CECARRYIN,cout,CARRYOUT_reg);

end
else begin
        reg_plus_mux   #(.PIPELINE(CARRYOUTREG),.WIDTH(1),.RST_TYPE(RSTTYPE)) reg_carro (CLK,RSTCARRYIN ,CECARRYIN,cout,CARRYOUT_reg);
end
```

```
////////////////////////////////////////////

if (OPMODEREG==0) begin
    reg_plus_mux   #(.PIPELINE(OPMODEREG),.WIDTH(8),.RST_TYPE(RSTTYPE)) reg_op (CLK,RSTOPMODE,CEOPMODE,OPMODE,OPMODE_reg);

end
else begin
        reg_plus_mux   #(.PIPELINE(OPMODEREG),.WIDTH(8),.RST_TYPE(RSTTYPE)) reg_op (CLK,RSTOPMODE ,CEOPMODE,OPMODE,OPMODE_reg);
end
endgenerate
////////////////////////////////////////////////////
always @(*) begin
if(B_INPUT=="DIRECT") begin
    B_mux=B;
end
else   if (B_INPUT=="CASCADE")begin
    B_mux=BCIN;
end
else begin
    B_mux=0;
end
//////////////////////////////
if(OPMODE_reg[6]==0) begin
 B_D_ADD_SUB=B_reg+D_reg;
end
else begin
    B_D_ADD_SUB=D_reg-B_reg;
end
//////////////////////////////
if(OPMODE_reg[4]==0) begin
    op_mode_4_mux=B_reg;
end
else begin
    op_mode_4_mux=B_D_ADD_SUB;
end
//////////////////////////
MUL= B_reg2*A_reg2;
 ////////////////
 case (OPMODE_reg[1:0])
2'b00: MUX_x=0;
2'b01:MUX_x={12'b0,M_reg};
2'b10:MUX_x=P_reg;
2'b11: MUX_x={D_reg[11:0],A_reg2[17:0],B_reg2[17:0]};
endcase
////////////////////////////////////
if(CARRYINSEL=="OPMODE5") begin
    CARRYIN_mux=OPMODE_reg[5];
end
else  if (CARRYINSEL==CARRYIN)begin
    CARRYIN_mux=CARRYIN;
end
else begin
    CARRYIN_mux=0;
end
```

```
220    ////////////////////////////////////////
221    case (OPMODE_reg[3:2])
222    2'b00: MUX_z=0;
223    2'b01:MUX_z=PCIN;
224    2'b10:MUX_z=P_reg;
225    2'b11: MUX_z=C_reg;
226    endcase
227    ////////////////////////////////////
228    if(OPMODE_reg[7]==1) begin
229    {cout,z_x_ADDER}=MUX_z-(MUX_x+CARRYIN_reg);
230    end
231    else begin
232        {cout,z_x_ADDER}=MUX_z+MUX_x+CARRYIN_reg;
233    end
234    end
235    endmodule
```

2. Tb:

```
1    module DSP_tb();
2    reg [17:0] A;
3    reg [17:0] B;
4    reg [47:0] C;
5    reg [17:0] D;
6    reg CARRYIN;
7    wire [35:0] M;
8    wire [47:0] P;
9    wire CARRYOUT;
10   wire CARRYOUTF;
11   reg CLK;
12   reg [7:0]OPMODE;
13   reg CEA,CEB,CEC,CECARRYIN,CED,CEM,CEOPMODE,CEP;
14   reg RSTA,RSTB,RSTC,RSTCARRYIN,RSTD,RSTM,RSTOPMODE,RSTP;
15   reg[47:0] PCIN;
16   wire [47:0] PCOUT;
17   wire [17:0] BCOUT;
18   reg [17:0] BCIN;
19   ////////////////////////////////////////////////////////////////////////
20   parameter A0REG=0;
21   parameter A1REG=1;
22   parameter B0REG=0;
23   parameter B1REG=1;
24   parameter CREG=1;
25   parameter DREG=1;
26   parameter MREG=1;
27   parameter PREG=1;
28   parameter CARRYINREG=1;
29   parameter CARRYOUTREG=1;
30   parameter OPMODEREG=1;
31   parameter CARRYINSEL="OPMODE5";
32   parameter B_INPUT="DIRECT";
33   parameter RSTTYPE="SYNC";
34   ////////////////////////////////////////////////////////
35   reg [17:0]W1;
36   reg [47:0] W2,W3;
37   reg [48:0] W4;
38    DSP  dsp (
39    A,
40    B,
41    C,
42    D,
43    CARRYIN,
44     M,
45    P,
46    CARRYOUT,
47    CARRYOUTF,
48    CLK,
49    OPMODE,
50    CEA,CEB,CEC,CECARRYIN,CED,CEM,CEOPMODE,CEP,
51    RSTA,RSTB,RSTC,RSTCARRYIN,RSTD,RSTM,RSTOPMODE,RSTP,
52     PCIN,
53    PCOUT,
54    BCOUT,
55    BCIN
```

```
56        );
57     ////////////////////////////
58     initial begin
59        CLK=0;
60        forever
61        #1 CLK=~CLK;
62     end
63     ////////////////////////////////
64     integer i;
65     initial begin
66     //rst test
67    RSTA=1; RSTB=1; RSTC=1; RSTD=1;
68    RSTOPMODE=1; RSTCARRYIN=1;
69    RSTP=1; RSTM=1;
70    ////////////////////////////////
71    A=0; B=0; C=0; D=0;
72    BCIN=0;  PCIN=0;
73    CARRYIN=0;  OPMODE=0;
74    CEA=0;CEB=0;CEC=0;CECARRYIN=0;CED=0;CEM=0;CEOPMODE=0;CEP=0;
75    repeat (8) begin
76        @(negedge CLK);
77    end
78    ////////////////////////////////
79    RSTA=0; RSTB=0; RSTC=0; RSTD=0;
80    RSTOPMODE=0; RSTCARRYIN=0;
81    RSTP=0; RSTM=0;
82    CEA=1;CEB=1;CEC=1;CECARRYIN=1;CED=1;CEM=1;CEOPMODE=1;CEP=1;
83    // test first adder (add), second adder (add) // 12
84    A=18'd1; B=18'd1 ; C= 48'd1; D=18'd1;
85    PCIN=48'd10;
86    OPMODE[6]=1'b0;
87    OPMODE[4]=1'b1;
88    OPMODE[1:0] =2'b01;
89    OPMODE[3:2]=2'b01;
90    OPMODE[7]=1'b0;
91    OPMODE[5]=1'b0;
92    W1=B+D;
93    W2=W1*A;
94    W3=W2+PCIN;
95    repeat(4) begin
96        @(negedge CLK);
97    end
98    if (P!=W3)
99        begin
100           $display ("error");
101           $display("P=%d,W3=%d",P,W3);
102           $stop;
103       end
104       else begin
105   $display("success");
106   end
107   ////////////////////////////////////////////
108   //test first adder (sub) , second adder (add) //14
109   A=18'd1; B=18'd1 ; C= 48'd1; D=18'd5;
110   PCIN=48'd10;
```

```
    OPMODE[6]=1'b1;
    OPMODE[4]=1'b1;
    OPMODE[1:0] =2'b01;
    OPMODE[3:2]=2'b01;
    OPMODE[7]=1'b0;
    OPMODE[5]=1'b0;
    W1=D-B;
    W2=W1*A;
    W3=W2+PCIN;
    repeat(4) begin
        @(negedge CLK);
    end
    if (P!=W3)
        begin
            $display ("error");
            $display("P=%d,W3=%d",P,W3);
            $stop;
        end
        else begin
    $display("success");
    end
    ////////////////////////////////////////
    //test second adder (sub) , first adder (add) //8
    A=18'd1; B=18'd1 ; C= 48'd1; D=18'd1;
    PCIN=48'd10;
    OPMODE[6]=1'b0;
    OPMODE[4]=1'b1;
    OPMODE[1:0] =2'b01;
    OPMODE[3:2]=2'b01;
    OPMODE[7]=1'b1;
    OPMODE[5]=1'b0;
    W1=D+B;
    W2=W1*A;
    W3=PCIN-W2;
    repeat(4) begin
        @(negedge CLK);
    end
    if (P!=W3)
        begin
            $display ("error");
            $display("P=%d,W3=%d",P,W3);
            $stop;
        end
        else begin
    $display("success");
    end
    ////////////////////////////////
    //test second adder (sub) , first adder (sub) //1
    A=18'd1; B=18'd1 ; C= 48'd1; D=18'd10;
    PCIN=48'd10;
    OPMODE[6]=1'b1;
    OPMODE[4]=1'b1;
    OPMODE[1:0] =2'b01;
    OPMODE[3:2]=2'b01;
    OPMODE[7]=1'b1;
```

```
166     OPMODE[5]=1'b0;
167     W1=D-B;
168     W2=W1*A;
169     W3=PCIN-W2;
170     repeat(4) begin
171         @(negedge CLK);
172     end
173     if (P!=W3)
174         begin
175             $display ("error");
176             $display("P=%d,W3=%d",P,W3);
177             $stop;
178         end
179         else begin
180     $display("success");
181     end
182     ////////////////////////////////////////////
183     //test second adder (add) , first adder (add),opmode_4_mux=B  // ((B*A)+PCIN)=11
184     A=18'd1; B=18'd1 ; C= 48'd1; D=18'd10;
185     PCIN=48'd10;
186     OPMODE[6]=1'b0;
187     OPMODE[4]=1'b0;
188     OPMODE[1:0] =2'b01;
189     OPMODE[3:2]=2'b01;
190     OPMODE[7]=1'b0;
191     OPMODE[5]=1'b0;
192     W2=B*A;
193     W3=W2+PCIN;
194     repeat(4) begin
195         @(negedge CLK);
196     end
197     if (P!=W3)
198         begin
199             $display ("error");
200             $display("P=%d,W3=%d",P,W3);
201             $stop;
202         end
203         else begin
204     $display("success");
205     end
206     ////////////////////////////////////////////////////
207     //test second adder (add) , first adder (add) , cin  // ((D+B)*A)+PCIN+CIN=22;
208     A=18'd1; B=18'd1 ; C= 48'd1; D=18'd10;
209     PCIN=48'd10;
210     OPMODE[6]=1'b0;
211     OPMODE[4]=1'b1;
212     OPMODE[1:0] =2'b01;
213     OPMODE[3:2]=2'b01;
214     OPMODE[7]=1'b0;
215     OPMODE[5]=1'b1;
216     W1=D+B;
217     W2=W1*A;
218     W3=W2+PCIN;
219     W4=W3+OPMODE[5];
220     repeat(4) begin
```

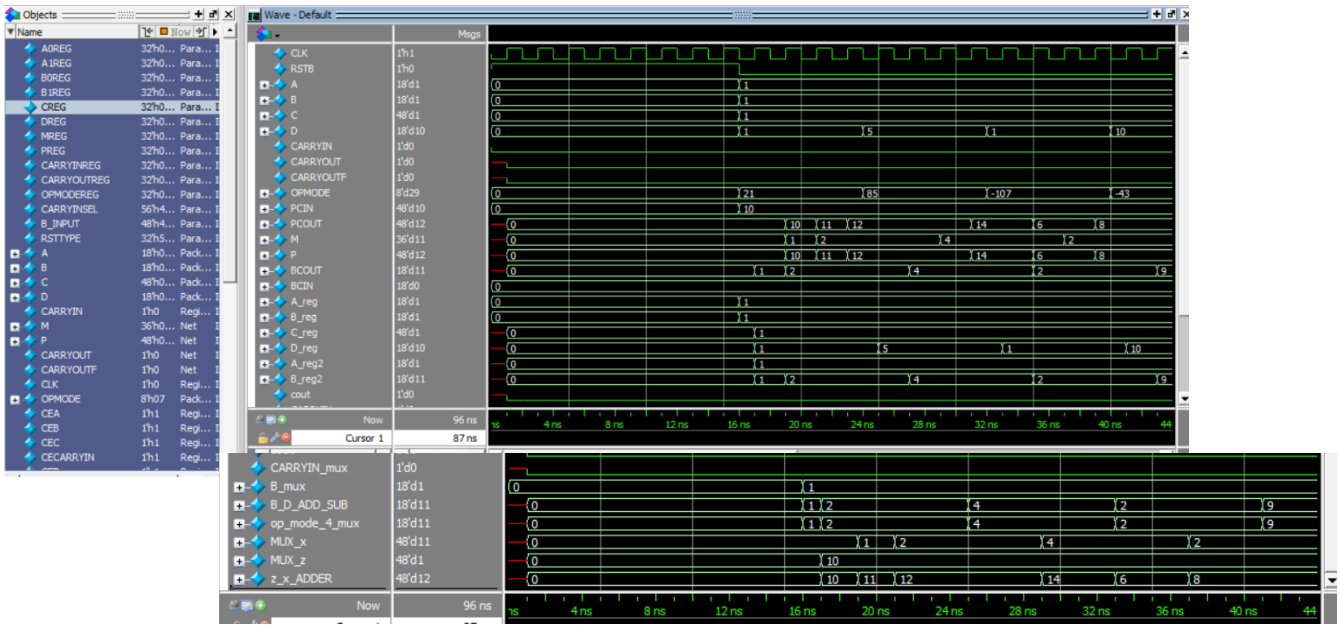
```
    @(negedge CLK);
end
if (P!=W4)
    begin
        $display ("error");
        $display("P=%d,W3=%d",P,W3);
        $stop;
    end
    else begin
$display("success");
end
////////////////////////////////////////////////
//test second adder (add) , first adder (add) , p as input  // P+PCIN=;
A=18'd1; B=18'd1 ; C= 48'd1; D=18'd10;
PCIN=48'd10;
OPMODE[6]=1'b0;
OPMODE[4]=1'b1;
OPMODE[1:0] =2'b10;
OPMODE[3:2]=2'b01;
OPMODE[7]=1'b0;
OPMODE[5]=1'b0;

repeat(4) begin
W3=P+PCIN;
    @(negedge CLK);

end
if (P!=W3)
    begin
        $display ("error");
        $display("P=%d,W3=%d",P,W3);
        $stop;
    end
    else begin
$display("success");
end
////////////////////////////////////////////////////////////////////////
//test second adder (add) , first adder (add) , select is 0 in mux x and z
A=18'd1; B=18'd1 ; C= 48'd1; D=18'd10;
PCIN=48'd10;
OPMODE[6]=1'b0;
OPMODE[4]=1'b1;
OPMODE[1:0] =2'b00;
OPMODE[3:2]=2'b00;
OPMODE[7]=1'b0;
OPMODE[5]=1'b0;

repeat(4) begin
    @(negedge CLK);
end
if (P!=0)
        begin
        $display ("error");
        $display("P=%d,W3=%d",P,W3);
        $stop;
```

```
276         end
277         else begin
278     $display("success");
279     end
280     ////////////////////////////////
281     //test second adder (add) , first adder (add) , select is  11 in mux z
282     //((D+B)*A)+C
283     A=18'd1; B=18'd1 ; C= 48'd1; D=18'd10;
284     PCIN=48'd10;
285     OPMODE[6]=1'b0;
286     OPMODE[4]=1'b1;
287     OPMODE[1:0] =2'b01;
288     OPMODE[3:2]=2'b11;
289     OPMODE[7]=1'b0;
290     OPMODE[5]=1'b0;
291     W1=D+B;
292     W2=W1*A;
293     W3=W2+C;
294     repeat(4) begin
295         @(negedge CLK);
296     end
297     if (P!=W3)
298         begin
299             $display ("error");
300             $display("P=%d,W3=%d",P,W3);
301             $stop;
302         end
303         else begin
304     $display("success");
305     end
306     ////////////////////////////////////////////////////////
307     // X sel = 3 [D:A:B] && Z sel = 01 [0]
308         A = 18'd1; B = 18'd1 ; C = 48'd1; D = 18'd1;
309         OPMODE[6] = 1'b0;
310         OPMODE[4] = 1'b0;
311         OPMODE[1:0] = 2'b11;
312         OPMODE[3:2] = 1'b01;
313         OPMODE[7] = 1'b0; // Add
314         OPMODE[5] = 1'b0;
315     W2={D[11:0],A[17:0],B[17:0]};
316     W3=W2+PCIN;
317         repeat(4) begin
318         @(negedge CLK);
319     end
320     if (P!=W3)
321         begin
322             $display ("error");
323             $display("P=%d,W3=%d",P,W3);
324             $stop;
325         end
326         else begin
327     $display("success");
328     end
329     $stop;
330     end
331     endmodule
332
```

3. Do file:

```
vlib work
vlog project_1.v reg+mux.v DSP_tb.v
vsim -voptargs=+acc work.DSP_tb
add wave *
run -all
#quit -sim
```

4. Waveform:



5. Transcript:

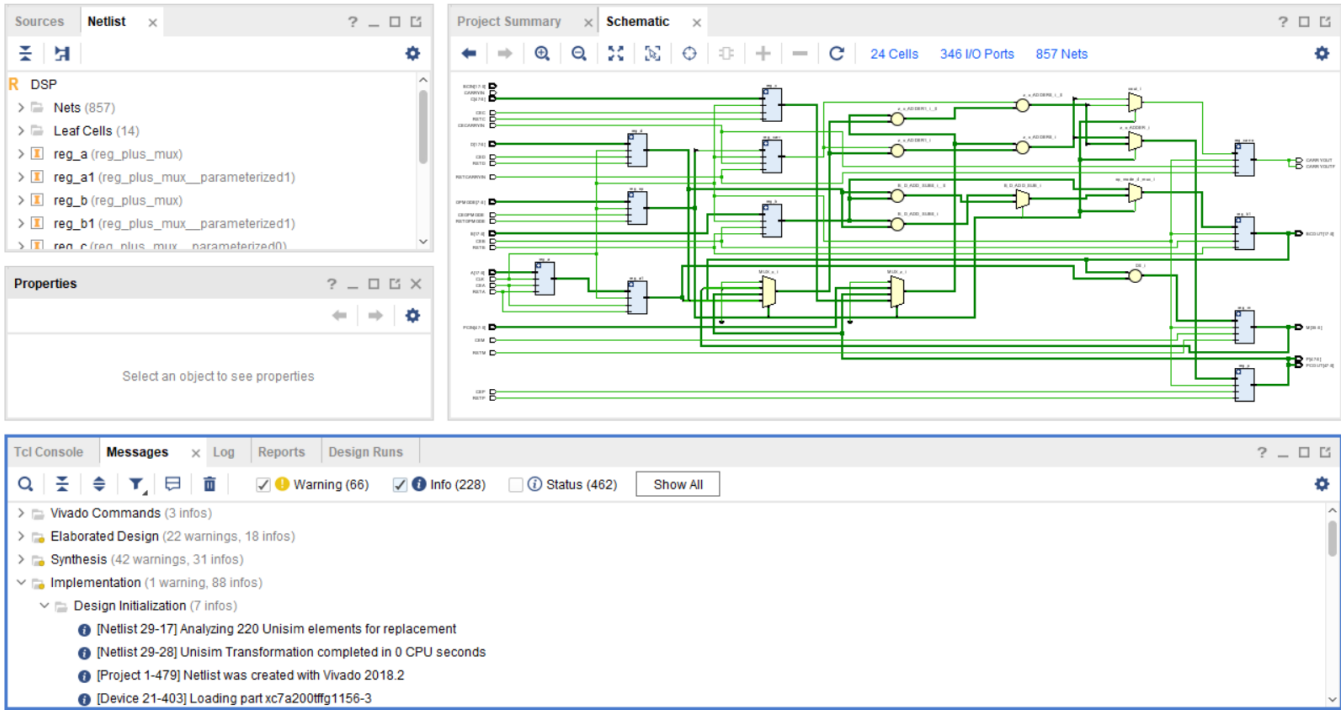
```
VSIM 44> run -all
# success
# success
# success
# success
# success
# success
# success
# success
# success
# success
```

Vivado and synthesis part:

1. Constrains file:

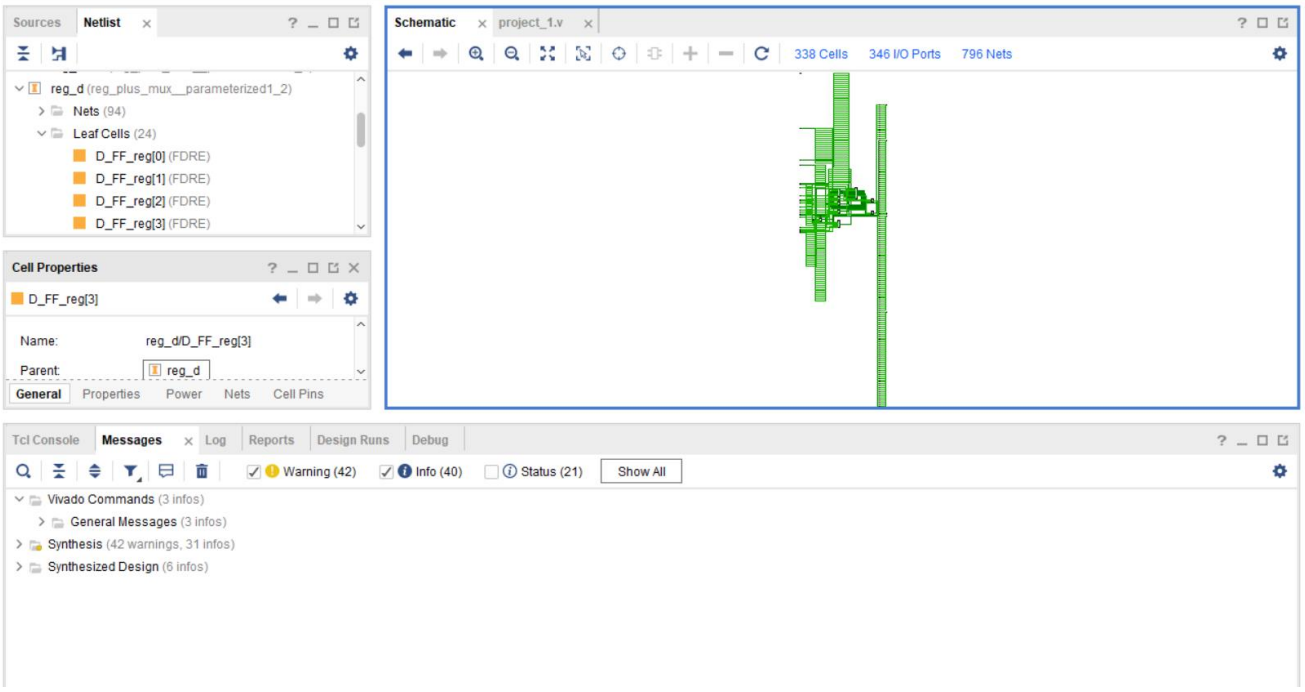
```
4   ## - rename the used ports (in each line, after get_ports) according to the top level signal names in
5
6   ## Clock signal
7   set_property -dict { PACKAGE_PIN W5   IOSTANDARD LVCMOS33 } [get_ports CLK]
8   create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports CLK]
9
10
11  ## Switches
12  #set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports {in0[0]}]
13  #set_property -dict { PACKAGE_PIN V16   IOSTANDARD LVCMOS33 } [get_ports {in0[1]}]
14  #set_property -dict { PACKAGE_PIN W16   IOSTANDARD LVCMOS33 } [get_ports {in1[0]}]
15  #set_property -dict { PACKAGE_PIN W17   IOSTANDARD LVCMOS33 } [get_ports {in1[1]}]
```

2. Elaboration:



3. Synthesis:

- **Schematic:**



- **Report utilization:**

Hierarchy

Name	Slice LUTs (134600)	Slice Registers (269200)	DSPs (740)	Bonded IOB (500)	BUFGCTRL (32)
DSP	255	160	1	327	1
reg_a1 (reg_plus_mux...	0	18	0	0	0
reg_b1 (reg_plus_mux...	0	18	0	0	0
reg_c (reg_plus_mux_...	0	48	0	0	0

- **Timing report:**

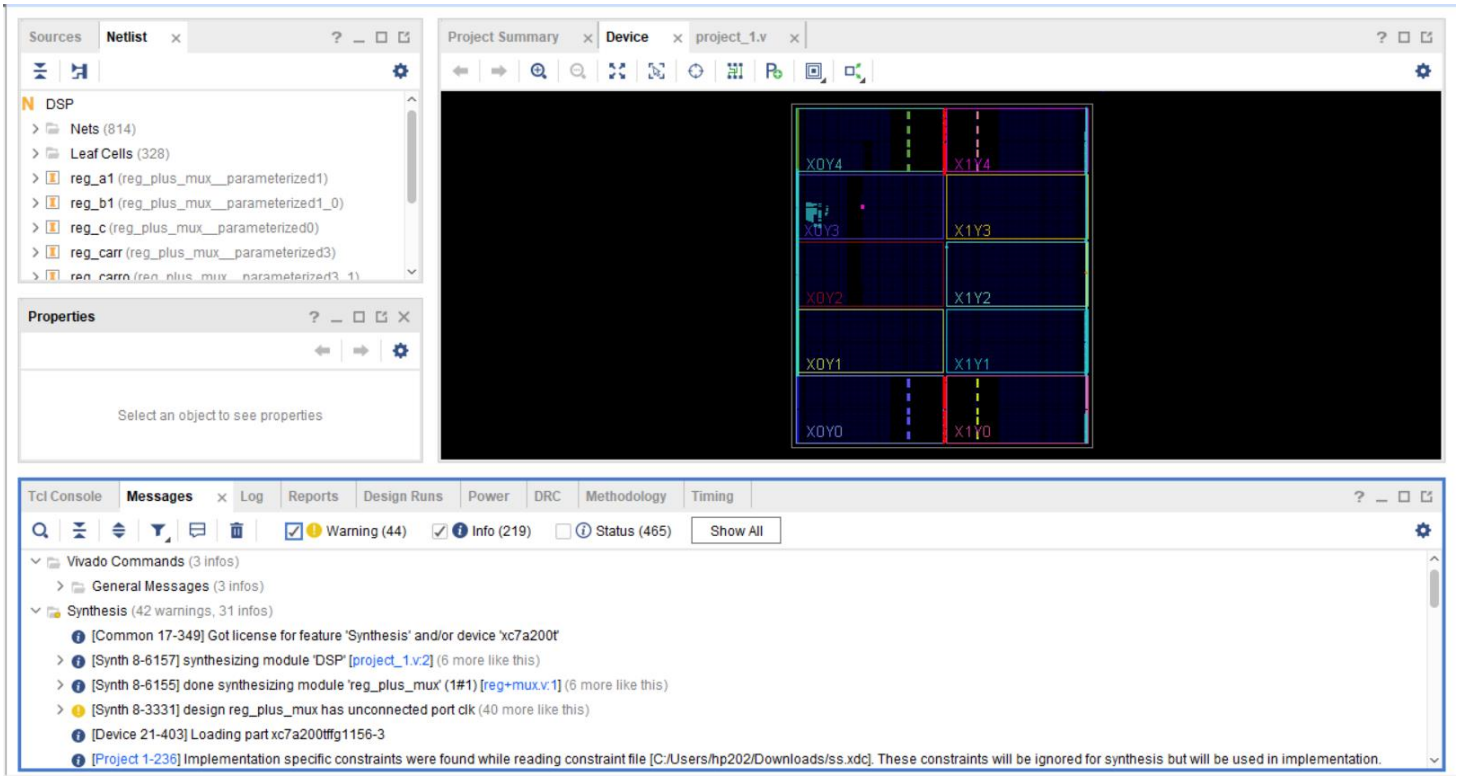
Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	5.216 ns	Worst Hold Slack (WHS):	0.182 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	106	Total Number of Endpoints:	106	Total Number of Endpoints:	162

All user specified timing constraints are met.

4. Implementation:

- **Schematic:**



- **Timing:**

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	3.713 ns	Worst Hold Slack (WHS):	0.273 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	125	Total Number of Endpoints:	125	Total Number of Endpoints:	181

All user specified timing constraints are met.

- **Utilization:**

Name	Slice LUTs (133800)	Slice Registers (267600)	Slice (33450)	LUT as Logic (133800)	LUT Flip Flop Pairs (133800)	DSPs (740)	Bonded IOB (500)	BUFGCTRL (32)
DSP	254	179	106	254	26	1	327	1
reg_a1 (reg_plus_mux...	0	18	6	0	0	0	0	0
reg_b1 (reg_plus_mux...	0	36	12	0	0	0	0	0
reg_c (reg_plus_mux_...	0	48	14	0	0	0	0	0